ISE Project Coding Standards

Team Guidelines

November 10, 2025

Quick Reference Table

Practice	Guideline
Naming	`lowerCamelCase` for all variables, functions, files, and class names. Example: `handleUserInput`, `chatbot.cpp`
Function Comments	Every function has a 2–4 line comment block describing *purpose*, parameters (`@param`), return value (`@return`), and any edge details.
Header Guards	Every header file (`.h`) must use `#ifndef`, `#define`, `#endif` format with a unique macro.
Formatting	Use 4 spaces for indentation. Place opening braces on the same line as the loop or function.
Commit Messages	Each commit starts with story/task code, type, then message (see Section). Allowed types: `add`, `fix`, `test`.
Code Review	Review all code before push; comment using `TODO:` and `FIXME:` for improvements.

Function Comment Example

```
/*
 * Finds and prints the chatbot's response for a given user string.
 * @param input: the user's typed message.
 * @return: true if a response is found, false otherwise.
 */
```

Commit Message Format

Type	When to Use	Example Commit Message
`add`	New code, features, or files	DP-01 [ADD]: implement chatbot input loop
`fix`	Bug fixes and corrections	DP-02 [FIX]: correct input matching logic
`test`	Adding new/manual/automated tests	DP-03 [TEST]: add test cases for file loading

Organizing Code: Header vs. Source Files

To keep our C++ codebase readable, maintainable, and easy for everyone to contribute, follow these practices:

- **Header files (.h):**
  - Write only **class/struct definitions** (data members and function signatures, not function bodies).
  - These files describe the public interface: what the class is and what it offers.
  - Add short comments above each class and public function, explaining purpose, key parameters (`@param`), and return value (`@return`).
- **Source files (.cpp):**
  - Provide the actual code for the functions declared in the header.
  - Use inline or block comments inside functions when logic is tricky.
  - Always start by including the matching header (e.g., `#include "fileio.h"`).
- **Why this split?** Keeps projects organized, speeds up teamwork, and prevents accidental code duplication. Changing a function's logic won't require recompiling all code that uses the interface.

`std` Namespaces and Best Practice:

- **Never put** `using namespace std;` in a header file.
  - If you don't like repeating `std::`, you can write `using std::string;` or similar only at the top of your .cpp files.
  - In headers, explicitly use `std::` prefix for types like `std::string`, `std::vector`, `std::unordered_map`, etc.
- This avoids hidden bugs and naming conflicts in your project.

Summary Table:

File Type	What to put inside	Where to comment
.h Header	Class/struct declarations, function signatures	Above each class/method declaration
.cpp Source	Function bodies (implementations)	Inside functions, especially on tricky logic

Always keep interfaces (`.h`) and implementations (`.cpp`) separate. Standard comments in headers make it easy for teammates to use your code.

Extra Tips

- Comment unfinished/problematic code with `TODO:` or `FIXME:`
- Keep functions focused: one main purpose, keep short when possible.
- Group helpers and logic by feature, not one file per tiny function.
- Always review your code and commit with intention!

Standardization makes teamwork simple — stick to these for an A+ workflow!